

AWS Cloud Handbook

1 AWS Handbook — Master Table of Contents

- [1.1 Part I — Cloud Foundations \(Pages 1–80\)](#)
- [1.2 Part II — Identity & Access Management \(Pages 81–160\)](#)
- [1.3 Part III — Networking \(Pages 161–280\)](#)
- [1.4 Part IV — Compute \(Pages 281–400\)](#)
- [1.5 Part V — Storage \(Pages 401–480\)](#)
- [1.6 Part VI — Databases \(Pages 481–560\)](#)
- [1.7 Part VII — Application Integration \(Pages 561–620\)](#)
- [1.8 Part VIII — DevOps on AWS \(Pages 621–700\)](#)
- [1.9 Part IX — Observability & Operations \(Pages 701–760\)](#)
- [1.10 Part X — Security & Compliance \(Pages 761–800\)](#)
- [1.11 Expansion status](#)

2 Chapter 1: Introduction to AWS & the Shared Responsibility Model

- [2.1 1.1 What is Amazon Web Services \(AWS\)?](#)
 - [2.1.1 Why AWS matters for DevOps](#)
- [2.2 1.2 Cloud service models](#)
 - [2.3 1.3 The Shared Responsibility Model](#)
 - [2.3.1 AWS is responsible for **Security OF the Cloud**](#)
 - [2.3.2 You are responsible for **Security IN the Cloud**](#)
 - [2.3.3 Example: RDS vs EC2 MySQL](#)
- [2.4 1.4 Core AWS concepts](#)
- [2.5 1.5 AWS console, CLI, and IaC](#)
- [2.6 1.6 Pricing fundamentals](#)
- [2.7 1.7 Well-Architected Framework \(preview\)](#)
- [2.8 1.8 Chapter summary](#)
- [2.9 Lab 1.1 — Create your AWS account](#)
- [2.10 Review questions](#)

3 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

- [3.1 2.1 Global infrastructure overview](#)
- [3.2 2.2 Regions](#)
 - [3.2.1 Choosing a region](#)
 - [3.2.2 Regional vs global services](#)
- [3.3 2.3 Availability Zones \(AZs\)](#)
 - [3.3.1 Multi-AZ patterns](#)
- [3.4 2.4 Local Zones, Wavelength, and Outposts](#)
- [3.5 2.5 Edge locations and CloudFront](#)
- [3.6 2.6 Fault isolation design principles](#)
 - [3.6.1 S3 durability vs availability](#)
- [3.7 2.7 Service quotas \(limits\)](#)
- [3.8 2.8 Chapter summary](#)
- [3.9 Lab 2.1 — Explore regions in CLI](#)
- [3.10 Review questions](#)

4 Chapter 3: AWS Account Setup, Billing, and Cost Basics

- [4.1 3.1 Creating a secure AWS account](#)
 - [4.1.1 Step-by-step](#)
 - [4.1.2 Root account checklist](#)
- [4.2 3.2 AWS Free Tier](#)
- [4.3 3.3 Billing and Cost Management](#)
 - [4.3.1 Budget example](#)
- [4.4 3.4 Cost optimization fundamentals](#)
- [4.5 3.4 Consolidated billing with Organizations](#)
- [4.6 3.5 Support plans \(overview\)](#)
- [4.7 3.6 Chapter summary](#)

[4.8 Lab 3.1](#)

[5 Chapter 5: IAM Users, Groups, Roles, and Policies](#)

[5.1 5.1 What is IAM?](#)

[5.2 5.2 IAM identity types](#)

[5.3 5.3 Policies — the core of permissions](#)

[5.3.1 Policy structure](#)

[5.3.2 Policy types](#)

[5.4 5.4 Users and groups](#)

[5.4.1 Create a developer group \(example workflow\)](#)

[5.5 5.5 Roles and trust policies](#)

[5.5.1 EC2 instance role example](#)

[5.5.2 Assume role \(cross-account or CLI\)](#)

[5.6 5.6 AWS managed vs customer managed policies](#)

[5.7 5.7 Least privilege in practice](#)

[5.7.1 Tag-based access \(ABAC\)](#)

[5.8 5.8 Common IAM actions for DevOps](#)

[5.9 5.9 IAM Access Analyzer & CloudTrail](#)

[5.10 5.10 Chapter summary](#)

[5.11 Lab 5.1 — IAM hands-on](#)

[5.12 Review questions](#)

[6 Chapter 6: IAM Best Practices and Permission Boundaries](#)

[6.1 6.1 The IAM security baseline](#)

[6.2 6.2 Prefer roles over users for applications](#)

[6.2.1 CI/CD role pattern](#)

[6.3 6.3 Permission boundaries](#)

[6.4 6.4 Policy writing tips](#)

[6.4.1 Start with AWS managed, refine to customer managed](#)

[6.4.2 Use IAM Policy Simulator](#)

[6.4.3 Avoid these anti-patterns](#)

[6.5 6.5 Service-linked roles](#)

[6.6 6.6 Cross-account access](#)

[6.7 6.7 Chapter summary](#)

[6.8 Lab 6.1](#)

[7 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables](#)

[7.1 9.1 What is a VPC?](#)

[7.2 9.2 CIDR and IP planning](#)

[7.2.1 Example VPC design](#)

[7.3 9.3 Subnets](#)

[7.4 9.4 Route tables](#)

[7.5 9.5 Internet Gateway and NAT \(preview\)](#)

[7.6 9.6 Security Groups vs NACLs](#)

[7.7 9.7 Terraform VPC snippet](#)

[7.8 9.8 Chapter summary](#)

[7.9 Lab 9.1](#)

[8 Chapter 10: Internet Gateway, NAT Gateway, and Routing](#)

[8.1 10.1 Internet Gateway \(IGW\)](#)

[8.2 10.2 NAT Gateway](#)

[8.2.1 NAT Gateway facts](#)

[8.3 10.3 VPC Endpoints — avoid NAT costs](#)

[8.4 10.4 Elastic IP \(EIP\)](#)

[8.5 10.5 Troubleshooting connectivity](#)

[8.5.1 Useful commands](#)

[8.6 10.6 High availability networking](#)

[8.7 10.7 Chapter summary](#)

[8.8 Lab 10.1](#)

[9 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates](#)

[9.1 15.1 Amazon EC2 overview](#)

[9.2 15.2 Instance families](#)

[9.2.1 Purchasing options](#)

[9.3 15.3 AMIs and launch templates](#)

[9.4 15.4 User data — bootstrap scripts](#)

[9.5 15.5 Auto Scaling Groups \(ASG\)](#)

[9.6 15.6 EC2 best practices](#)

[9.7 15.7 Chapter summary](#)

[9.8 Lab 15.1](#)

10 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle

[10.1 20.1 Amazon S3 overview](#)

[10.2 20.2 Core concepts](#)

[10.3 20.3 Storage classes](#)

[10.4 20.4 Versioning and lifecycle](#)

[10.4.1 Versioning](#)

[10.4.2 Lifecycle rules](#)

[10.5 20.5 Security](#)

[10.5.1 Block Public Access \(account and bucket level\)](#)

[10.5.2 Encryption](#)

[10.5.3 Bucket policies vs ACLs](#)

[10.6 20.6 Static website hosting](#)

[10.7 20.7 S3 performance tips](#)

[10.8 20.8 Chapter summary](#)

[10.9 Lab 20.1](#)

11 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards

[11.1 35.1 Amazon CloudWatch overview](#)

[11.2 35.2 Metrics](#)

[11.2.1 View metrics](#)

[11.2.2 Custom metrics](#)

[11.3 35.3 Alarms](#)

[11.3.1 Alarm states](#)

[11.3.2 Composite alarms](#)

[11.4 35.4 Dashboards](#)

[11.5 35.5 CloudWatch Logs \(overview\)](#)

[11.6 35.6 Observability best practices on AWS](#)

[11.7 35.7 Chapter summary](#)

[11.8 Lab 35.1](#)

1 AWS Handbook — Master Table of Contents

Target length: ~800 pages (~320,000 words)
Audience: Beginner → Advanced AWS practitioners
Companion: [DevOps Handbook](#)

1.1 Part I — Cloud Foundations (Pages 1-80)

Ch	Title	Pages
1	Introduction to AWS & Shared Responsibility	1-18
2	AWS Global Infrastructure: Regions, AZs, Edge	19-35
3	AWS Account Setup, Billing, and Cost Basics	36-55
4	AWS CLI, CloudShell, and SDK Overview	56-80

Written: Chapters 1-3, 5-6, 9-10, 15, 20, 35 (11 chapters + full TOC)

1.2 Part II — Identity & Access Management (Pages 81-160)

Ch	Title	Pages
5	IAM Users, Groups, Roles, and Policies	81-105
6	IAM Best Practices & Permission Boundaries	106-125
7	IAM Identity Center (SSO) & Federation	126-145
8	AWS Organizations & Service Control Policies	146-160

Written: [Chapter 5](#)

1.3 Part III — Networking (Pages 161-280)

Ch	Title	Pages
9	VPC Fundamentals: CIDR, Subnets, Route Tables	161-185
10	Internet Gateway, NAT, and Routing	186-205
11	Security Groups vs NACLs	206-220
12	Load Balancers: ALB, NLB, GWLB	221-245
13	Route 53: DNS Records and Routing Policies	246-265
14	VPC Peering, Transit Gateway, PrivateLink	266-280

1.4 Part IV — Compute (Pages 281-400)

Ch	Title	Pages
----	-------	-------

15	EC2: Instance Types, AMIs, Launch Templates	281–305
16	Auto Scaling & Elastic Load Balancing Integration	306–325
17	Lambda: Events, Concurrency, and Layers	326–350
18	ECS & Fargate	351–375
19	EKS: Managed Kubernetes on AWS	376–400

1.5 Part V — Storage (Pages 401–480)

Ch	Title	Pages
20	S3: Buckets, Objects, Versioning, Lifecycle	401–425
21	S3 Security, Encryption, and Static Websites	426–445
22	EBS & EFS	446–465
23	AWS Backup & Disaster Recovery Patterns	466–480

1.6 Part VI — Databases (Pages 481–560)

Ch	Title	Pages
24	RDS & Aurora	481–505
25	DynamoDB: Tables, Indexes, Streams	506–525
26	ElastiCache & Caching Patterns	526–540
27	Database Migration (DMS) & DR	541–560

1.7 Part VII — Application Integration (Pages 561–620)

Ch	Title	Pages
28	API Gateway	561–580
29	SQS, SNS, and EventBridge	581–605
30	Step Functions Workflows	606–620

1.8 Part VIII — DevOps on AWS (Pages 621–700)

Ch	Title	Pages
31	CloudFormation Deep Dive	621–645
32	AWS CDK	646–665
33	CodePipeline, CodeBuild, CodeDeploy	666–685
34	Systems Manager & Parameter Store	686–700

1.9 Part IX — Observability & Operations (Pages 701-760)

Ch	Title	Pages
35	CloudWatch Metrics, Alarms, Dashboards	701-720
36	CloudWatch Logs & Insights	721-735
37	X-Ray & Distributed Tracing	736-750
38	CloudTrail & AWS Config	751-760

1.10 Part X — Security & Compliance (Pages 761-800)

Ch	Title	Pages
39	KMS, Secrets Manager, ACM	761-775
40	GuardDuty, Security Hub, WAF, Shield	776-790
41	Well-Architected Review & Capstone Design	791-800

1.11 Expansion status

Part	Status
I-II, III-V, IX	Ch 1-3, 5-6, 9-10, 15, 20, 35 written (~120 pages of content)

To request a chapter: *“Expand AWS Handbook Chapter 15 (EC2 deep dive)”*

2 Chapter 1: Introduction to AWS & the Shared Responsibility Model

AWS Handbook — Part I, Pages 1–18

2.1 1.1 What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

2.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
 - **Global scale** — Deploy in multiple regions for resilience and latency.
 - **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
 - **Ecosystem** — Largest cloud market share; extensive documentation and community.
-

2.2 1.2 Cloud service models

Model	You manage	Provider manages	AWS examples
IaaS	OS, apps, data	Hardware, network, hypervisor	EC2, VPC, EBS
PaaS	Apps, data	Runtime, OS, infra	Elastic Beanstalk, App Runner
SaaS	Configuration, data	Everything else	WorkSpaces, Chime
FaaS	Function code	Everything else	Lambda

Containers blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

2.3 1.3 The Shared Responsibility Model

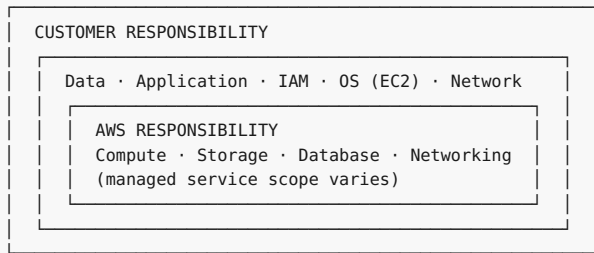
Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

2.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

2.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)
- Network configuration (security groups, NACLs)
- Application code security



2.3.3 Example: RDS vs EC2 MySQL

Task	EC2 + self-managed MySQL	Amazon RDS
Patch MySQL	You	AWS
Patch OS	You	AWS
Configure security groups	You	You
Encrypt data at rest	You (configure)	You (enable)
Physical security	AWS	AWS

2.4 1.4 Core AWS concepts

Concept	Definition
Region	Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code>)
Availability Zone (AZ)	Isolated data center within a region
Account	Billing and security boundary (12-digit ID)
Resource	Any AWS entity (EC2 instance, S3 bucket, etc.)
ARN	Amazon Resource Name — unique identifier
Tag	Key-value metadata on resources

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

2.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

2.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1–3 year commitment; lower cost.
- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

Always set billing alarms (AWS Budgets) on new accounts.

2.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
 2. Security
 3. Reliability
 4. Performance Efficiency
 5. Cost Optimization
 6. Sustainability
-

2.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
 - **Shared responsibility** splits security between AWS and you—know your side.
 - Learn **regions, accounts, ARNs, and tags** early.
 - Use CLI and IaC for production; console for exploration.
-

2.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
 2. Enable **MFA** on the root account; do not use root daily.
 3. Create an **IAM admin user** for daily work.
 4. Set a **billing budget** alert at \$10 or your comfort threshold.
 5. Explore the console: EC2, S3, IAM dashboards.
-

2.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
 2. Who patches the OS on an EC2 instance?
 3. What is an ARN used for?
 4. Name two AWS pricing models.
-

Next: [Chapter 2 — Global Infrastructure](#)

3 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

AWS Handbook — Part I, Pages 19–35

3.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

Region (e.g., us-east-1)

- Availability Zone A (us-east-1a)
- Availability Zone B (us-east-1b)
- Availability Zone C (us-east-1c)
- ... (typically 3+ AZs per region)

Edge Locations (200+) — CloudFront, Route 53, etc.

3.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

3.2.1 Choosing a region

Factor	Guidance
Latency	Place resources near users
Compliance	Data residency (GDPR, etc.) may require EU regions
Service availability	New services launch in select regions first
Cost	Pricing varies by region (us-east-1 often cheapest)
Disaster recovery	Use a second region for critical workloads

Naming: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

3.2.2 Regional vs global services

Regional (must specify region)	Global (single endpoint)
EC2, RDS, VPC, Lambda	IAM, Route 53, CloudFront
S3 (bucket tied to region)	AWS Organizations

3.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like us-east-1a are **account-specific mappings**— 1a in your account may not be the same physical DC as 1a in another account.

3.3.1 Multi-AZ patterns

Pattern	Use case
Active-passive	RDS Multi-AZ failover
Active-active	ALB spanning AZs with EC2 in each
Stretch cluster	EKS node groups per AZ

3.4 2.4 Local Zones, Wavelength, and Outposts

Type	Purpose
Local Zones	Extend region closer to metro areas (ultra-low latency)
Wavelength	5G edge at carrier locations
Outposts	AWS hardware on-premises; hybrid cloud

These are specialized; start with standard regions + AZs.

3.5 2.5 Edge locations and CloudFront

Edge locations cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

3.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

3.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

3.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

3.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.

- Deploy production workloads across **multiple AZs**.
 - **Edge locations** power CloudFront and global services.
 - Know **regional vs global** services when designing architecture.
-

3.9 Lab 2.1 — Explore regions in CLI

```
aws configure                                # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

3.10 Review questions

1. How many AZs should a highly available web app use at minimum?
 2. Is IAM a regional or global service?
 3. What AWS service uses edge locations for caching?
 4. Why might you choose `eu-central-1` over `us-east-1`?
-

Next: Part II — [Chapter 5: IAM Fundamentals](#)

4 Chapter 3: AWS Account Setup, Billing, and Cost Basics

AWS Handbook — Part I, Pages 36–55

4.1 3.1 Creating a secure AWS account

4.1.1 Step-by-step

1. **Sign up** at <https://aws.amazon.com> with a dedicated email (not personal if possible for work).
2. **Root account** — Full admin access. Enable **MFA immediately**.
3. **Do not use root** for daily tasks.
4. Create an **IAM admin user** or use **IAM Identity Center** for workforce access.
5. Set up **billing alerts** before launching resources.

4.1.2 Root account checklist

- ☐ MFA enabled (hardware key preferred)
 - ☐ Strong unique password in password manager
 - ☐ No access keys on root
 - ☐ Contact info and alternate contacts set
 - ☐ AWS Budgets alert configured
-

4.2 3.2 AWS Free Tier

Type	Examples
12-month free	750 hrs/month t2/t3.micro EC2, 5 GB S3, RDS limits
Always free	Lambda 1M requests/month, DynamoDB 25 GB
Short trials	Some services have limited trials

Warning: Free tier expires; resources left running can incur charges. Always set billing alarms.

4.3 3.3 Billing and Cost Management

Tool	Purpose
Billing Dashboard	Current month spend
Cost Explorer	Historical analysis, forecasts, filters
AWS Budgets	Email/SNS alerts at thresholds
Cost & Usage Report (CUR)	Granular export to S3 for analysis
Pricing Calculator	Estimate architecture costs before building

4.3.1 Budget example

Create a monthly budget of \$20 with alerts at 50%, 80%, and 100%:

```
aws budgets create-budget \
  --account-id 123456789012 \
  --budget file://budget.json \
  --notifications-with-subscribers file://notifications.json
```

4.4 3.4 Cost optimization fundamentals

1. **Tag everything** — Environment, Project, Owner, CostCenter
2. **Right-size** — Use Compute Optimizer recommendations
3. **Stop dev resources** — Schedule EC2/RDS shutdown nights/weekends
4. **Use Spot** for fault-tolerant batch workloads
5. **S3 lifecycle** — Move old data to IA/Glacier
6. **Reserved capacity** — 1–3 year Savings Plans for steady workloads
7. **Delete unused** — EBS volumes, Elastic IPs, old snapshots

4.5 3.4 Consolidated billing with Organizations

AWS Organizations lets you manage multiple accounts under one payer account:

- **Consolidated billing** — Single invoice, volume discounts
- **SCPs** — Guardrails on what accounts can do
- **Separate accounts** per environment (dev/staging/prod) or per team

Recommended for any serious AWS usage.

4.6 3.5 Support plans (overview)

Plan	Best for
Basic	Free; account and billing support
Developer	Business hours email support
Business	24/7 phone, < 1 hr response for production down
Enterprise	TAM, < 15 min critical response

4.7 3.6 Chapter summary

- Secure the **root account** with MFA; use IAM for daily work.
- Set **budgets and alerts** on day one.
- **Tag** resources and review Cost Explorer weekly during learning.

4.8 Lab 3.1

1. Enable MFA on root.
2. Create IAM admin user `admin-yourname`.
3. Create a \$10 monthly budget with email alert.
4. Enable Cost Explorer (may take 24h for data).

Next: [Chapter 6 — IAM Best Practices](#)

5 Chapter 5: IAM Users, Groups, Roles, and Policies

AWS Handbook — Part II, Pages 81–105

5.1 5.1 What is IAM?

AWS Identity and Access Management (IAM) controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access).

IAM is **global**—users and roles exist at the account level, not per region.

5.2 5.2 IAM identity types

Identity	Use case
Root user	Account owner; created at signup. Avoid for daily use.
IAM user	Long-term credentials for humans or legacy apps
IAM role	Temporary credentials; assumed by users, services, or other accounts
IAM group	Collection of users with shared permissions

Best practice: Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

5.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

5.3.1 Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

```
}
```

5.3.2 Policy types

Type	Attached to
Identity-based	Users, groups, roles
Resource-based	S3 buckets, SNS topics, etc.
Permission boundaries	Max permissions for identity
SCP	AWS Organizations — account ceiling
Session policies	Limit role session scope

Evaluation logic: Explicit **Deny** always wins. Default is implicit deny until Allow.

5.4 5.4 Users and groups

5.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/PowerUserAccess
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
```

5.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

5.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

5.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/DeployRole \
  --role-session-name deploy-session
```

5.6 5.6 AWS managed vs customer managed policies

Type	Description
AWS managed	Maintained by AWS (<code>ReadOnlyAccess</code> , <code>AdministratorAccess</code>)
Customer managed	You create and reuse across identities
Inline	Embedded in single user/group/role; harder to reuse

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

5.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

5.7.1 Tag-based access (ABAC)

```
"Condition": {
  "StringEquals": {
    "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"
  }
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

5.8 5.8 Common IAM actions for DevOps

Action prefix	Service
<code>ec2:*</code>	EC2
<code>s3:*</code>	S3
<code>iam:PassRole</code>	Allow attaching roles to services
<code>sts:AssumeRole</code>	Switch roles
<code>cloudformation:*</code>	Stack operations

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

5.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

5.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
 - Use **groups** for humans, **roles** for services and temporary access.
 - Policies are JSON with Effect, Action, Resource, Condition.
 - Apply **least privilege**; avoid root and long-lived access keys.
-

5.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
 2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
 3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
 4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
 5. Review effective permissions in **IAM Policy Simulator**.
-

5.12 Review questions

1. What is the difference between an IAM user and an IAM role?
 2. What happens when Allow and Deny conflict?
 3. Why should EC2 instances use roles instead of access keys?
 4. What does `sts:AssumeRole` do?
-

Continue: Chapter 6 — IAM Best Practices (outline in TOC)

6 Chapter 6: IAM Best Practices and Permission Boundaries

AWS Handbook — Part II, Pages 106–125

6.1 6.1 The IAM security baseline

Follow these rules in every AWS account:

1. **Root account** — MFA only; no access keys; break-glass use only
2. **No long-lived access keys** for humans — use SSO + temporary credentials
3. **Least privilege** — Grant minimum permissions required
4. **MFA** — Required for console access and sensitive API calls
5. **Rotate keys** — If keys must exist, rotate every 90 days max
6. **Audit** — CloudTrail enabled in all regions; review Access Analyzer findings

6.2 6.2 Prefer roles over users for applications

Identity	Use for
IAM user + keys	Legacy; avoid for new projects
IAM role	EC2, Lambda, ECS, cross-account, CI/CD
IAM Identity Center	Human workforce access

6.2.1 CI/CD role pattern

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com" },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:*"
      }
    }
  }]
}
```

GitHub Actions assumes this role—no static AWS keys in GitHub Secrets.

6.3 6.3 Permission boundaries

A **permissions boundary** sets the **maximum** permissions an identity can have—even if policies grant more.

Use for:

- Delegating IAM admin to developers without full account access

- Sandbox accounts with caps

```
aws iam put-user-permissions-boundary \  
  --user-name developer \  
  --permissions-boundary arn:aws:iam::aws:policy/PowerUserAccess
```

6.4 6.4 Policy writing tips

6.4.1 Start with AWS managed, refine to customer managed

```
{  
  "Version": "2012-10-17",  
  "Statement": [{  
    "Sid": "DeployToSpecificBucket",  
    "Effect": "Allow",  
    "Action": ["s3:PutObject", "s3:GetObject"],  
    "Resource": "arn:aws:s3::my-app-artifacts-prod/*"  
  }]  
}
```

6.4.2 Use IAM Policy Simulator

Test policies before deploying: AWS Console → IAM → Policy Simulator.

6.4.3 Avoid these anti-patterns

- "Action": "*" on production roles
- "Resource": "*" when ARNs are known
- Attaching AdministratorAccess to CI roles
- Sharing one IAM user across a team

6.5 6.5 Service-linked roles

AWS creates **service-linked roles** for services (e.g., AWSServiceRoleForElasticLoadBalancing). Do not delete these unless AWS docs say it's safe.

6.6 6.6 Cross-account access

Pattern: Account A (workload) trusts Account B (CI/CD) via role assumption.

```
# From CI in account B  
aws sts assume-role \  
  --role-arn arn:aws:iam::WORKLOAD_ACCOUNT:role/DeployRole \  
  --role-session-name github-deploy
```

Use **AWS Organizations** SCPs to prevent privilege escalation across accounts.

6.7 6.7 Chapter summary

- Humans: **SSO + MFA**. Apps: **roles**. CI: **OIDC federation**.
- Write **scoped customer-managed policies**; test with Policy Simulator.
- Use **permission boundaries** and **SCPs** for guardrails.

6.8 Lab 6.1

1. Create a CI/CD role trust policy for GitHub OIDC (or a second AWS account).
2. Attach a least-privilege policy for S3 deploy only.
3. Run Access Analyzer and remediate any `Public` findings.

Next: [Chapter 9 — VPC Fundamentals](#)

7 Chapter 9: VPC Fundamentals — CIDR, Subnets, and Route Tables

AWS Handbook — Part III, Pages 161–185

7.1 9.1 What is a VPC?

A **Virtual Private Cloud (VPC)** is your isolated network in AWS. You control IP ranges, subnets, routing, and firewalls.

Every AWS account has a **default VPC** per region (with public subnets). Production workloads use **custom VPCs** with explicit design.

7.2 9.2 CIDR and IP planning

CIDR (Classless Inter-Domain Routing) defines IP ranges:

CIDR	Addresses	Usable (AWS reserves 5)
10.0.0.0/16	65,536	~65,531
10.0.1.0/24	256	~251
10.0.1.0/28	16	~11

7.2.1 Example VPC design

```
VPC: 10.0.0.0/16

Public subnets (ALB, NAT Gateway):
  10.0.1.0/24 - us-east-1a
  10.0.2.0/24 - us-east-1b

Private subnets (app servers, ECS, Lambda):
  10.0.10.0/24 - us-east-1a
  10.0.11.0/24 - us-east-1b

Database subnets (RDS, no internet):
  10.0.20.0/24 - us-east-1a
  10.0.21.0/24 - us-east-1b
```

Tip: Plan for growth; changing VPC CIDR later is painful. Use /16 for VPC, /24 for subnets.

7.3 9.3 Subnets

Subnets live in **one Availability Zone**. Types:

Type	Route to IGW	Typical workloads
Public	Yes (via route table)	ALB, NAT Gateway, bastion
Private	No direct IGW	App servers, workers
Isolated	No NAT either	Databases

`map_public_ip_on_launch = true` assigns public IPs to instances in public subnets.

7.4 9.4 Route tables

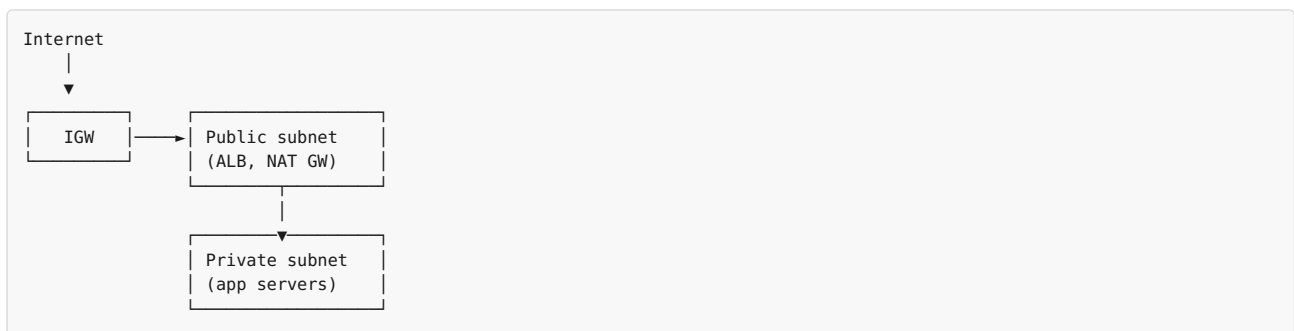
A **route table** defines where traffic goes:

Destination	Target	Meaning
10.0.0.0/16	local	Traffic within VPC
0.0.0.0/0	igw-xxxxx	Internet via Internet Gateway
0.0.0.0/0	nat-xxxxx	Internet via NAT Gateway (private subnet)
pl-xxxxx	VPC Endpoint	Private AWS service access

Each subnet associates with **one** route table.

7.5 9.5 Internet Gateway and NAT (preview)

- **Internet Gateway (IGW)** — VPC component for bidirectional internet access
- **NAT Gateway** — Allows private subnet instances to reach internet (outbound only); place in public subnet



7.6 9.6 Security Groups vs NACLs

	Security Group	NACL
Level	Instance/ENI	Subnet
State	Stateful	Stateless
Rules	Allow only	Allow and deny
Default	Deny all inbound	Allow all

Best practice: Use security groups as primary firewall; NACLs for subnet-level deny rules if needed.

Example security group for web tier:

- Inbound: 443 from ALB security group
- Outbound: 5432 to database security group

7.7 9.7 Terraform VPC snippet

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support = true
  tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = true
}
```

7.8 9.8 Chapter summary

- Design VPCs with **public**, **private**, and **database** subnet tiers across **multiple AZs**.
- **Route tables** control internet and local routing.
- **Security groups** are stateful instance firewalls; default deny inbound.

7.9 Lab 9.1

1. Create a custom VPC with 2 public and 2 private subnets in different AZs.
2. Attach an Internet Gateway and configure route tables.
3. Launch an EC2 in a private subnet (no public IP) and verify no direct internet access.

Next: [Chapter 10 — NAT and Routing](#)

8 Chapter 10: Internet Gateway, NAT Gateway, and Routing

AWS Handbook — Part III, Pages 186–205

8.1 10.1 Internet Gateway (IGW)

An **Internet Gateway** is a horizontally scaled, redundant VPC component that allows:

- Instances with public IPs to communicate with the internet
- Internet to initiate connections to instances with public IPs

Attach one IGW per VPC (for internet-connected VPCs).

```
aws ec2 create-internet-gateway
aws ec2 attach-internet-gateway --internet-gateway-id igw-xxx --vpc-id vpc-xxx
```

Public subnet route table:

Destination	Target
0.0.0.0/0	igw-xxxxxxx
10.0.0.0/16	local

8.2 10.2 NAT Gateway

Private subnet instances should **not** have public IPs. They reach the internet (for updates, API calls) via a **NAT Gateway** in a public subnet.

```
Private subnet instance (10.0.10.5)
  → NAT Gateway (public subnet, elastic IP)
  → Internet Gateway
  → Internet
```

8.2.1 NAT Gateway facts

- **Managed** by AWS (no patching)
- **Charged** per hour + per GB processed
- **AZ-specific** — deploy one NAT per AZ for HA, or accept cross-AZ failover tradeoffs
- **Alternative:** NAT instances (self-managed, not recommended)

Private subnet route table:

Destination	Target
0.0.0.0/0	nat-xxxxxxx
10.0.0.0/16	local

8.3 10.3 VPC Endpoints — avoid NAT costs

For AWS service traffic (S3, DynamoDB, etc.), use **VPC endpoints** to keep traffic on the AWS network—no NAT charges.

Type	Services	Cost
Gateway	S3, DynamoDB	Free
Interface	Most other services	Per hour + data

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id      = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  route_table_ids = [aws_route_table.private.id]
}
```

8.4 10.4 Elastic IP (EIP)

A **static public IPv4** address. Required for NAT Gateway.

- **Charge** applies if EIP is allocated but not attached to a running instance
- Release unused EIPs to avoid fees

8.5 10.5 Troubleshooting connectivity

Problem	Check
Can't SSH to public instance	SG allows 22; has public IP; IGW attached; correct route
Private instance can't reach internet	NAT in public subnet; private RT has 0.0.0.0/0 → NAT
Can't reach S3 from private subnet	VPC endpoint or NAT route
Inter-subnet blocked	NACL rules; SG rules

8.5.1 Useful commands

```
aws ec2 describe-route-tables --filters "Name=vpc-id,Values=vpc-xxx"
aws ec2 describe-network-interfaces --filters "Name=subnet-id,Values=subnet-xxx"
# From instance:
curl -s https://checkip.amazonaws.com # Public IP test
```

8.6 10.6 High availability networking

Production checklist:

- ☐ Multi-AZ subnets
- ☐ NAT Gateway per AZ (or accept risk)
- ☐ ALB spans multiple AZs
- ☐ VPC endpoints for S3/DynamoDB/ECR/API
- ☐ No single-AZ dependencies for critical paths

8.7 10.7 Chapter summary

- **IGW** enables public internet for public subnets.

- **NAT Gateway** provides outbound-only internet for private subnets.
 - Use **VPC endpoints** to reduce NAT costs and improve security for AWS APIs.
-

8.8 Lab 10.1

1. Deploy NAT Gateway in public subnet with EIP.
 2. Configure private subnet route table.
 3. Launch EC2 in private subnet; `curl` an external API successfully.
 4. Add S3 gateway endpoint; verify S3 access without NAT.
-

Next: [Chapter 15 — EC2](#)

9 Chapter 15: EC2 — Instance Types, AMIs, and Launch Templates

AWS Handbook — Part IV, Pages 281–305

9.1 15.1 Amazon EC2 overview

Elastic Compute Cloud (EC2) provides resizable virtual machines in the cloud. You choose:

- **Instance type** — CPU, memory, network, storage
- **AMI** — Operating system image
- **Networking** — VPC, subnet, security groups
- **Storage** — EBS volumes
- **IAM role** — Permissions without access keys

9.2 15.2 Instance families

Family	Use case	Examples
T (burstable)	Dev, low-traffic web	t3.micro, t3.small
M (general)	Balanced workloads	m6i.large
C (compute)	CPU-intensive	c6i.xlarge
R (memory)	Databases, caches	r6i.large
G/P (GPU)	ML, graphics	g5.xlarge
I (storage)	High IOPS databases	i3.large

Naming: `m6i.large` = family **m**, gen **6**, Intel **i**, size **large**.

9.2.1 Purchasing options

Option	Cost	Interruption
On-Demand	Highest	Never
Reserved / Savings Plans	30–70% less	Never
Spot	Up to 90% less	Can be terminated with 2-min notice
Dedicated Host	Compliance	Never

9.3 15.3 AMIs and launch templates

AMI (Amazon Machine Image) — Template for root volume: OS + optional software.

Sources:

- AWS-provided (Amazon Linux 2023, Ubuntu)
- Marketplace
- Custom (built with EC2 Image Builder or Packer)

Launch Template — Reusable EC2 configuration (instance type, AMI, SG, user data, IAM role). Preferred over launch configurations.

```
aws ec2 run-instances \  
  --image-id ami-0c55b159cbfafelf0 \  
  --instance-type t3.micro \  
  --key-name my-key \  
  --security-group-ids sg-xxx \  
  --subnet-id subnet-xxx \  
  --iam-instance-profile Name=EC2-S3-ReadRole \  
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-01}]'
```

9.4 15.4 User data — bootstrap scripts

Run commands on first boot:

```
#!/bin/bash  
yum update -y  
yum install -y nginx  
systemctl enable nginx  
systemctl start nginx  
echo "<h1>Hello from $(hostname)</h1>" > /usr/share/nginx/html/index.html
```

Pass via console, CLI `--user-data`, or launch template. Logs: `/var/log/cloud-init-output.log`.

9.5 15.5 Auto Scaling Groups (ASG)

ASG maintains desired instance count and replaces unhealthy instances.

```
resource "aws_autoscaling_group" "web" {  
  name                = "web-asg"  
  vpc_zone_identifier = [aws_subnet.public_a.id, aws_subnet.public_b.id]  
  target_group_arns   = [aws_lb_target_group.web.arn]  
  health_check_type   = "ELB"  
  min_size            = 2  
  max_size            = 10  
  desired_capacity    = 2  
  
  launch_template {  
    id       = aws_launch_template.web.id  
    version = "$Latest"  
  }  
}
```

Pair with **ALB** for traffic distribution and **scaling policies** on CPU or request count.

9.6 15.6 EC2 best practices

- Use **IAM roles**, never embed access keys
 - **Session Manager** instead of SSH where possible (no open port 22)
 - **IMDSv2** required (`HttpTokens: required`) — prevents SSRF credential theft
 - **Encrypted EBS** volumes by default
 - **Patch** via SSM Patch Manager or golden AMI pipelines
 - **Right-size** with CloudWatch metrics and Compute Optimizer
-

9.7 15.7 Chapter summary

- Choose instance **family and size** based on workload profile.
 - Use **launch templates** and **user data** for repeatable deployments.
 - **ASG + ALB** provides scalable, self-healing web tiers.
-

9.8 Lab 15.1

1. Launch Amazon Linux 2023 t3.micro with nginx user data.
 2. Attach IAM role for S3 read.
 3. Create launch template and ASG (min 1, max 2) behind an ALB.
-

Next: [Chapter 20 — S3](#)

10 Chapter 20: S3 — Buckets, Objects, Versioning, and Lifecycle

AWS Handbook — Part V, Pages 401–425

10.1 20.1 Amazon S3 overview

Simple Storage Service (S3) is object storage: store files (objects) in containers (buckets). Virtually unlimited scale; 99.999999999% durability.

Use cases:

- Static websites and assets
- Data lakes and backups
- Application uploads
- Log archives
- Terraform state files

10.2 20.2 Core concepts

Concept	Description
Bucket	Global unique name; lives in a region
Object	File + metadata; up to 5 TB
Key	Object path (e.g., <code>images/photo.jpg</code>)
Prefix	Simulated folders via key prefixes

```
aws s3 mb s3://my-unique-bucket-name-12345
aws s3 cp report.pdf s3://my-bucket/reports/2026/
aws s3 ls s3://my-bucket/reports/
aws s3 sync ./local-dir s3://my-bucket/backup/
aws s3 rm s3://my-bucket/old/ --recursive
```

10.3 20.3 Storage classes

Class	Use case	Retrieval
Standard	Frequent access	Immediate
Standard-IA	Infrequent access	Immediate, min storage charge
Glacier Instant	Archive, instant retrieval	Immediate
Glacier Flexible	Archive	Minutes to hours
Glacier Deep Archive	Long-term archive	12–48 hours
Intelligent-Tiering	Unknown access patterns	Auto-moves tiers

10.4 20.4 Versioning and lifecycle

10.4.1 Versioning

Protects against accidental deletes; keeps all versions of objects.

```
aws s3api put-bucket-versioning \
  --bucket my-bucket \
  --versioning-configuration Status=Enabled
```

10.4.2 Lifecycle rules

Automatically transition or expire objects:

```
{
  "Rules": [{
    "ID": "archive-old-logs",
    "Status": "Enabled",
    "Filter": { "Prefix": "logs/" },
    "Transitions": [{
      "Days": 30,
      "StorageClass": "STANDARD_IA"
    }, {
      "Days": 90,
      "StorageClass": "GLACIER"
    }],
    "Expiration": { "Days": 365 }
  }]
}
```

10.5 20.5 Security

10.5.1 Block Public Access (account and bucket level)

Enable by default on all buckets unless hosting a public static site intentionally.

10.5.2 Encryption

Method	Description
SSE-S3	AWS-managed keys
SSE-KMS	KMS keys; audit trail
SSE-C	Customer-provided keys
Client-side	Encrypt before upload

10.5.3 Bucket policies vs ACLs

- **Bucket policies** (JSON) — Preferred; resource-based IAM
- **ACLs** — Legacy; avoid for new projects

Example: allow CloudFront OAC to read objects (not public internet).

10.6 20.6 Static website hosting

```
# Enable static website hosting on bucket
aws s3 website s3://my-bucket/ --index-document index.html
```

Front with **CloudFront** + **ACM certificate** for HTTPS. Do not expose bucket directly to public.

10.7 20.7 S3 performance tips

- Use **multipart upload** for files > 100 MB
 - Prefix keys for parallelism (S3 scales automatically)
 - **S3 Transfer Acceleration** for long-distance uploads
 - **S3 Inventory** for auditing object metadata
-

10.8 20.8 Chapter summary

- S3 stores **objects in buckets** with global-unique names.
 - Enable **versioning** and **lifecycle** for data protection and cost control.
 - **Block public access**; use bucket policies and CloudFront for controlled delivery.
-

10.9 Lab 20.1

1. Create bucket with versioning and default encryption (SSE-S3).
 2. Upload files; delete one; restore previous version.
 3. Add lifecycle rule to transition `logs/` to IA after 30 days.
 4. Host static `index.html` behind CloudFront.
-

Next: [Chapter 35 — CloudWatch](#)

11 Chapter 35: CloudWatch — Metrics, Alarms, and Dashboards

AWS Handbook — Part IX, Pages 701–720

11.1 35.1 Amazon CloudWatch overview

CloudWatch is AWS’s observability service for **metrics**, **logs**, **alarms**, and **dashboards**. Nearly every AWS service emits CloudWatch metrics automatically.

11.2 35.2 Metrics

A **metric** is a time-series datapoint (e.g., `CPUUtilization` for EC2).

Concept	Description
Namespace	Grouping (AWS/EC2, AWS/S3, custom)
Metric name	e.g., <code>CPUUtilization</code>
Dimensions	InstanceId, FunctionName, etc.
Resolution	Standard 1 min; high-resolution down to 1 sec

11.2.1 View metrics

```
aws cloudwatch list-metrics --namespace AWS/EC2
aws cloudwatch get-metric-statistics \
  --namespace AWS/EC2 \
  --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=i-0abc123 \
  --start-time 2026-09-01T00:00:00Z \
  --end-time 2026-09-01T12:00:00Z \
  --period 300 \
  --statistics Average
```

11.2.2 Custom metrics

```
import boto3
cloudwatch = boto3.client('cloudwatch')
cloudwatch.put_metric_data(
    Namespace='MyApp/Orders',
    MetricData=[{
        'MetricName': 'OrdersProcessed',
        'Value': 42,
        'Unit': 'Count'
    }]
)
```

11.3 35.3 Alarms

Alarms trigger actions when metrics breach thresholds.

```
aws cloudwatch put-metric-alarm \
```

```
--alarm-name high-cpu-web-server \
--metric-name CPUUtilization \
--namespace AWS/EC2 \
--statistic Average \
--period 300 \
--threshold 80 \
--comparison-operator GreaterThanThreshold \
--evaluation-periods 2 \
--alarm-actions arn:aws:sns:us-east-1:123456789012:ops-alerts \
--dimensions Name=InstanceId,Value=i-0abc123
```

11.3.1 Alarm states

State	Meaning
OK	Within threshold
ALARM	Breached threshold
INSUFFICIENT_DATA	Not enough datapoints

11.3.2 Composite alarms

Combine multiple alarms with AND/OR logic to reduce alert noise.

11.4 35.4 Dashboards

Visualize metrics in the console or via API. Example Terraform:

```
resource "aws_cloudwatch_dashboard" "main" {
  dashboard_name = "production-overview"
  dashboard_body = jsonencode({
    widgets = [{
      type = "metric"
      width = 12
      height = 6
      properties = {
        metrics = [
          ["AWS/EC2", "CPUUtilization", "InstanceId", "i-0abc123"],
          [".", "NetworkIn", ".", "."]
        ]
      }
      period = 300
      stat = "Average"
      region = "us-east-1"
      title = "EC2 Web Server"
    }
  ])
}
```

11.5 35.5 CloudWatch Logs (overview)

Feature	Purpose
Log groups / streams	Organize application logs
Logs Insights	SQL-like query language
Subscription filters	Stream to Lambda, Kinesis, OpenSearch
Metric filters	Turn log patterns into metrics

```
aws logs tail /aws/lambda/my-function --follow
```

11.6 35.6 Observability best practices on AWS

1. **Alarm on symptoms** — Error rate, latency p99, not just CPU
 2. **SNS → PagerDuty/Slack** for on-call routing
 3. **Dashboards per service** + one executive overview
 4. **Log retention** — Set retention policies (cost control)
 5. **X-Ray** for distributed tracing (complements CloudWatch)
 6. Use **Container Insights** / **Lambda Insights** for managed runtimes
-

11.7 35.7 Chapter summary

- CloudWatch collects **metrics** from AWS and custom apps.
 - **Alarms** notify via SNS when thresholds breach.
 - **Dashboards** and **Logs** complete the operational picture.
-

11.8 Lab 35.1

1. Create SNS topic `ops-alerts` (email subscription).
 2. Create CPU alarm on an EC2 instance.
 3. Build a dashboard with EC2 CPU, ALB request count, and RDS connections.
 4. Run a Logs Insights query on a Lambda log group.
-

Continue: Chapter 36 — CloudWatch Logs Deep Dive (outline in TOC)